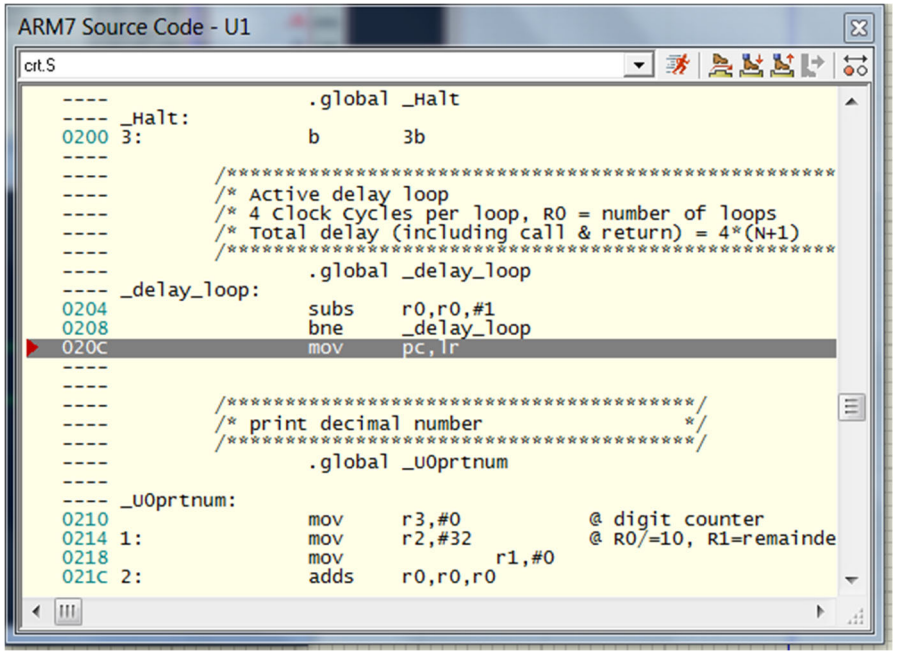
Notas sobre el microcontrolador LPC2103 (ARM7) en Proteus

Descomprimimos el fichero “starter_kit.zip” en un directorio del ordenador. Al hacer doble click en el fichero “ejemplo2103.vsmp”, el proyecto de trabajo se abre en el programa “VSM Studio”, el cual nos permitirá editar nuestro código fuente para el ARM7. Una vez compilemos el código, podremos simular su ejecución en un microcontrolador LPC2103 con el software comercial de diseño de circuitos “Proteus”.

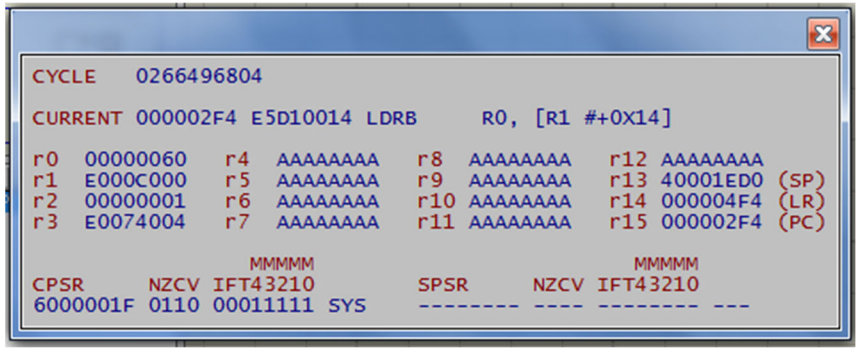
Compilar un proyecto

Pulsamos el botón para compilar y linkar el proyecto, asegurándonos de tener seleccionada la opción debug (Debug) para poder depurar el programa. El compilador creará una subcarpeta **Debug** en el directorio del proyecto donde introducirá los ficheros-objeto generados. Si no ha habido errores el compilador nos lo indicará y estaremos en condiciones de pulsar el icono para abrir el Proteus y simular el programa cargado en el LPC2103. El programa se ejecutará directamente. Con los controles situados en la parte inferior izquierda de la pantalla podremos pausar la simulación y proceder a su depurado. Para depurar el código, Proteus cuenta con una serie de ventanas que nos permiten visualizar el valor de las variables, los registros de la CPU, etc. Se puede elegir visualizar o no cada una de las ventanas seleccionándolas en el menú **Debug -> ARM7** de Proteus mientras la simulación está pausada. De este modo se puede visualizar la ventana Source Code (**Debug -> ARM7 -> Source Code)** en la que aparece la línea de código que se está ejecutando o cualquier otra parte del programa.



Utilizando los botones situados en la esquina superior derecha de esta pantalla podemos ejecutar paso a paso el programa, definir breakpoints (puntos de pausa), entrar/salir a/desde subrutinas, ejecutar paso a paso sin entrar en subrutinas, etc. Los números que aparecen a la izquierda del código (en azul) indican la dirección de memoria donde se encuentra cada instrucción. Nótese que siempre se trata de múltiplos de 4, lo que indica que las instrucciones se encuentran perfectamente alineadas y que ocupan 4 posiciones de memoria, es decir 4 bytes.

Otra ventana interesante es la ventana de registros de la CPU (**CPU registers**). Para visualizarla, seleccionarla en el menú **Debug -> ARM7 -> CPU Registers**.

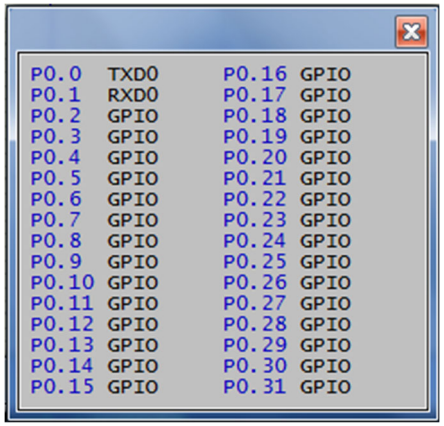


Ventana de registros de la CPU

En ella aparece en primer término el ciclo de reloj en el que se encuentra la ejecución del programa, la dirección donde está la instrucción que se va a ejecutar, dicha instrucción escrita en hexadecimal (8 cifras hex) y en lenguaje simbólico. A continuación, los valores (en hexadecimal) contenidos en los 16 registros pertenecientes al modo de funcionamiento en que nos encontramos (éste aparece en medio de la última línea, en la figura es el modo sistema SYS). Y en la última línea los campos significativos (flags, habilitación de interrupciones, estado Thumb, y el código del modo de funcionamiento) de los registros de estado actual (CPRS) y de salvaguarda (SPRS).

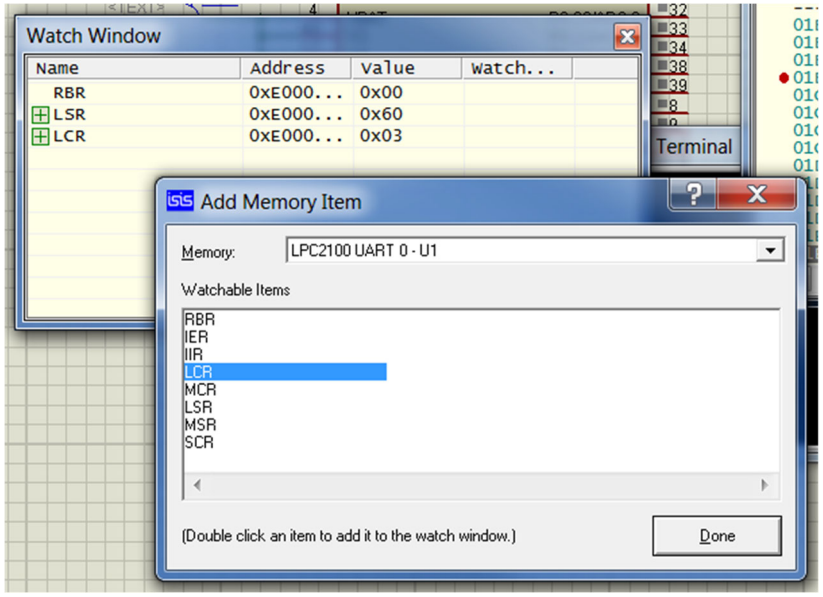
Otras ventanas del Debug de utilidad son:

- **Debug -> ARM7 -> Pin Configuration**. Indica la configuración seleccionada para los pines de input/output del micro. En nuestro caso se puede comprobar que los pines 1 y 2 del micro están conectados a la UART0 para visualizar caracteres en el terminal virtual.

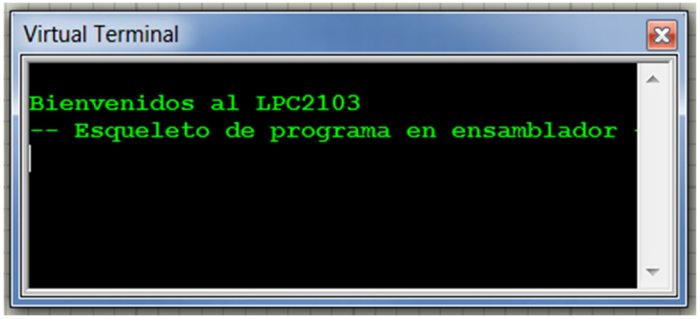


- **Debug -> ARM7 -> Flash**. Presenta los contenidos de la memoria Flash. En nuestro caso, puesto que el programa se ejecuta sobre la Flash, en esta memoria aparecerá el código. A esta memoria corresponden los 32 kbytes más bajos (0x0000 – 0x7FFF) de los 2^{32} direcciones de memoria a las que (en teoría) puede acceder el PC del ARM7.
- **Debug -> ARM7 -> RAM**. Presenta los contenidos de la memoria RAM. Se puede observar que esta memoria consta sólo de 8 Kbytes y que se le asigna las posiciones de memoria comprendidas entre 0x400000000 y 0x40001FFF. En nuestro caso, puesto que el programa que estamos ejecutando define y utiliza las pilas de cada modo en la RAM (como no puede ser de otro modo), se puede observar que contiene datos diferentes de AA (valor por defecto) en sus posiciones finales.
- **Debug -> Watch Window**. Para visualizar los valores de variables. En esta ventana también es posible visualizar los registros asociados a los periféricos. Éstos se pueden añadir por dirección (la dirección que les asigna la CPU para realizar operaciones de entrada/salida sobre ellos) o por nombre. Para visualizar un valor

del registro por nombre se coloca el ratón dentro de la ventana **Watch Window** y se presiona su botón derecho. Esto origina que se despliegue un menú donde escogemos la opción **Add Items (By name)**. En la ventana que aparece escogemos el periférico dentro del menú en el cuadro **Memory** y con esto en **Watchable Items** nos aparecen los registros relacionados con dicho periférico. Con doble "click" escogemos el registro concreto a visualizar. En la figura aparece cómo seleccionar el registro LCR de la UART0.



Si ahora pulsamos el botón otra vez eliminamos los breakpoints que hayamos insertado y pulsando el programa se ejecutará. En la pantalla del terminal virtual aparecerá el mensaje:



Subrutinas de entrada/salida

Para la realización de la primera práctica sobre el ARM7 utilizaremos los ficheros que se incluyen en el proyecto. En el fichero **crt.S** aparecen los vectores de interrupción, los programas de atención a las distintas fuentes de interrupción, las pilas y las subrutinas que ejecutándose en modo usuario están escritas en ensamblador. En este apartado nos referiremos a éstas últimas. En concreto, en el fichero **crt.s**, a partir de la línea 181 se encuentran las subrutinas de espera y de entrada/salida que podríamos necesitar en la realización de la práctica para la visualización del resultado en el terminal virtual conectado a la UART.

Para la elaboración de estas subrutinas se ha seguido la convención de paso de parámetros vista en clase.

Rutinas de entrada-salida en crt.S:

_Halt: No tiene parámetros. Se detiene la ejecución hasta un nuevo reset.

_U0putch: parámetro en R0: código ASCII del carácter a transmitir. Envía el carácter almacenado en R0 a través de la UART0 al hiperterminal.

_U0puts: parámetro: puntero al primer carácter de un texto (string) terminado en 0x00. Transmite por la UART0 la cadena de

caracteres ASCII apuntada por R0. La cadena debe terminar con un carácter nulo (ASCII 0x00).

_U0getch: sin parámetros. Espera recibir un dato por el receptor de la UART0. Cuando esto ocurre devuelve en R0 el código ASCII del carácter recibido. *_U0getch* no retorna hasta que no se recibe un dato.

***_U0prthex*:** parámetro de 32 bits en R0. Imprime en hexadecimal, a través de la UART0 el valor de R0 como un dato de 32 bits (8 dígitos hexadecimales).

_U0prthex8: parámetro de 8 bits en R0. Imprime en hexadecimal, a través de la UART0, el valor de R0 como un dato de 8 bits (2 dígitos hexadecimales).

***_U0prtnum*:** parámetro de 32 bits en R0. Imprime en decimal, a través de la UART0 el valor de R0. Se imprimen los dígitos justos para representar el valor (se eliminan ceros a la izquierda).

***_delay_loop*:** parámetro en R0. Rutina para hacer retardos. El tiempo que se tarda en retornar es (4*R0+4), medido en ciclos de reloj (16.95 ns por cada ciclo de reloj).

Ejemplo: subrutinas _U0prthex y _U0prthex8

Estas rutinas comparten código. Se encuentran situadas entre las líneas 235 y 254 del fichero **crt.S**. Aparecen, por tanto 3 etiquetas (_U0prthex:, _U0prthex8 y _U0prthex_entry) que indican tres entradas a la subrutina común a ambas.

Las dos subrutinas constan de un único parámetro que se encuentra en R0. Comenzaremos explicando la subrutina **_U0prthex**. Esta subrutina convierte este número de 32 bits en 8 cifras hexadecimales y las envía al puerto serie (UART0) para que puedan ser visualizados en el terminal virtual.

Comienza en la línea 236 cargando el registro R2 con el número 8. 8 es el número de cifras hexadecimales a visualizar. R2 se decrementará cada vez que salga una cifra nueva hasta llegar a 0, lo cual indicará que la subrutina ha finalizado.

En la siguiente línea se realiza un salto incondicional a la etiqueta **_U0prthex_entry**, donde comienza el código común con la subrutina **_U0prthex8**.

```
235 _U0prthex:          @ 32 bits
236         mov     r2,#8
237         b       _U0prthex_entry
```

El código continúa ejecutando la línea 242 donde se carga el registro R1 con la dirección donde se encuentra el registro THR de la UART. En este registro THR es donde hay que almacenar el byte (se interpreta como un carácter en ASCII) que se quiere enviar a la UART. A continuación, R3 se carga con el contenido de la dirección apuntada por R1+0x14, es decir, el contenido de la dirección 0xE000C014. En esta dirección se encuentra el registro LSR de la UART0 que se utiliza para informar a la CPU del estado en que se encuentra, es decir, si la UART0 está lista para recibir un dato desde la CPU o no. Esto se comprueba en la siguiente línea (línea 244), donde se compara el estado de la UART0 (recogido en R3) con el literal 0x20. Esta comparación se realiza con la instrucción tst que hace la función AND bit a bit de R3 con 0x20 (que es un número con todo 0 menos la 5ª posición, en la que hay un 1. Esta posición coincide con el bit de LSR que indica si la UART está lista para recibir datos o no. En la siguiente línea (línea 245) se comprueba el resultado de esta operación con beq (branch if equal = branch if Z=1). Mientras Z=1 (indica que el bit de LSR de la UART=0 => no está lista) seguiremos comprobando el estado, ya que saltamos a la etiqueta local 1,

donde volvemos a leer el registro LSR. En este bucle seguiremos hasta que Z=0, o lo que es lo mismo LSR indique que la UART está lista.

```
241  _U0prthex_entry:
242          ldr     r1,=0xE000C000   @ UART0 THR
243  1:      ldrb    r3,[r1,#0x14]    @ UART0 LSR, wait until THRE is one
244          tst     r3,#0x20
245          beq     1b
```

A partir de este momento comenzamos a convertir los 4 primeros bits (los más significativos, más a la izquierda) de R0 en la cifra hexadecimal correspondiente en ASCII. Para ello en la línea 246 rotamos a la derecha 28 posiciones r0, lo cual es equivalente a rotarlo 4 a la izquierda (en el ARM7 no existe rotación a la izquierda). Con esta operación en los bits más a la derecha de R0 aparecen los 4 que originariamente estaban más a la izquierda. Con la siguiente instrucción (línea 247) se aplica una máscara para aislar esos 4 bits y almacenarlos en otro registro (R3). Con esto, en R3 tenemos todo 0's y los 4 bits de la primera cifra hexadecimal en las cuatro posiciones menos significativas. En la siguiente línea comparamos ese número con 9, para saber si es mayor o menor que él, ya que en el código ASCII los números del 0 al 9 ocupan posiciones consecutivas y de la A a la F también, existiendo un "offset" entre el 9 y la A de 7 (ver código ASCII standard al final del documento). Si la cifra ASCII resulta ser mayor que 9, se suma 7 a R3 en la línea 249 y si no, se deja como está.

En la línea 250 se suma al número hexadecimal (con el offset si es el caso) el carácter ASCII del 0 [1] y con eso en R3 ya aparece el carácter ASCII correspondiente a la cifra hexadecimal. No queda más que enviarlo al registro THR de la UART, cosa que se hace en la línea 251.

```
242          ldr     r1,=0xE000C000   @ UART0 THR
243  1:      ldrb    r3,[r1,#0x14]    @ UART0 LSR, wait until THRE is one
244          tst     r3,#0x20
245          beq     1b
246          mov     r0,r0, ror #28   @ ...rol #4
247          and     r3,r0,#0x0f
248          cmp     r3,#9        @ 0..9 or A..F ?
249          addgt   r3,r3,#7     @ offset from '9'+1 to 'A' in the ASCII code
250          add     r3,r3,#'0'   @ convert to ASCII
251          strb    r3,[r1]      @ send through UART0 THR
```

Por último, se decrementa R2 (contador de cifras hexadecimales que inicialmente se carga con 8) y se salta a la etiqueta local 1: en el caso de que R2 sea distinto de 0 porque todavía no se ha acabado con todas las cifras hexadecimales. En caso contrario, es decir si R2=0, se acaba la subrutina y se vuelve al programa principal restaurando el valor del PC almacenado en LR.

```
252          subs    r2,r2,#1     @ repeat until no more digits pending
253          bne     1b
254          mov     pc,lr
```

La subrutina **_U0prthex8** funciona igual que la _U0prthex con la única diferencia de que convierte sólo un byte (2 cifras hexadecimales) situado en los 8 bits de más a la derecha del registro R0. La entrada a esta subrutina se encuentra en la línea 238 y en las dos líneas siguientes se carga R2 con el número 2 (en vez del 8) y se desplaza a la izquierda 24 bits el contenido

[1] Escribiendo el carácter ASCII entre apóstrofos (') nos estamos refiriendo al carácter ASCII correspondiente. No hace falta conocer la tabla de caracteres ASCII y escribir su equivalente en hexadecimal.

de R0 (línea 240) para que el byte a convertir se encuentre inicialmente en los 8 bits de más a la izquierda de R0.

Símbolos utilizados en Ensamblador

@ - comentario

/* */ - comentario en una línea completa

- operando inmediato (literal)

[ ] – dirección de memoria

0x – número hexadecimal

Etiq: - etiqueta

‘ ‘ – carácter en ASCII

Etiquetas globales – deben empezar por una letra y son sensibles a mayúsculas/minúsculas. Las etiquetas locales se nombran con números. Para definir una etiqueta como global y de este modo poder invocarla desde otros ficheros (en el linkado), hay que declararla con la directiva **.global _nombre_etiq**. Se definen como locales aquellas etiquetas comprendidas entre dos globales. En los saltos a etiquetas locales hay que indicar si éstas se encuentran delante o detrás de la instrucción de salto colocando una **f** (forward) o una **b** (backward) respectivamente a continuación del número (i.e. bne 1b)

Primeros Pasos

Vamos a escribir un programa muy sencillo en main para comprobar la función de las subrutinas comentadas.

El fichero **main.S** que se adjunta aparece a continuación:

```
1          .text
2          .global main
3  main:
4          stmfd   sp!,{lr}    @ salva LR por si retornamos
5          adr     r0,texto1
6          bl      _U0puts
7          ldmfd   sp!,{pc}    @ retornamos
8
9
10 texto1: .asciz "\r\nBienvenidos al LPC2103\r\n-- Esqueleto de programa en ensamblador --\r\n"
11
12         .align 2
13         .end
```

El programa main comienza guardando en la pila el contenido del registro LR (que contiene la dirección de retorno hacia la rutina de reset que lo invocó). De este modo, desde el programa main se puede llamar a subrutinas y no sobreescribir la dirección de retorno de main hacia la rutina de reset.

A continuación se introduce en el registro R0 la dirección de memoria donde comienza el string situado en la etiqueta texto1 usando la pseudoinstrucción ADR. En la siguiente línea se llama a la subrutina _U0puts para que el string aparezca en el terminal virtual. Por último retorna al programa que llamó a main lo que equivale a llamar a _Halt y detener la ejecución del programa.

Vamos a añadir en este mismo fichero una subrutina que nos permita visualizar por el hiperterminal el contenido de un registro (32 bits) en binario. Llamaremos a esta subrutina **U0prtbin.** Comenzamos colocando su etiqueta **U0prtbin:** (no hace falta definirla como global ya que la llamaremos desde el mismo fichero main.S). Esta subrutina llamará a su vez a la subrutina **_U0putch** que es la que se utilizará para visualizar los caracteres en ASCII. Por tener nuestra subrutina que llamar a otra, tendremos que guardar el contenido de R4 y R5 en la pila (con **stmfd**) y tras ejecutar la subrutina U0prtbin recuperarlo (con **ldmfd**). Estas serán la

primera y última de las líneas de la subrutina y su contenido se escribirá entre ellas. Escribimos en el fichero **main.S** entre las líneas **.align 2** y **.end** lo siguiente:

```
U0prtbin:
        stmfd   sp!, {r4, r5, lr}
        ldmfd   sp!, {r4, r5, pc}


.end
```

A continuación escribimos el contenido de la subrutina que, una vez completada, quedará como:

```
U0prtbin:
        stmfd   sp!, {r4, r5, lr}
        mov     r4, r0
        mov     r5, #32
1:      movs    r4, r4, lsl #1
        movcs   r0, #'1'
        movcc   r0, #'0'
        bl      _U0putch
        subs    r5, r5, #1
        bne     1b
        ldmfd   sp!, {r4, r5, pc}
```

Finalmente, añadimos algunas líneas en main que nos van a servir para comprobar si la subrutina U0prtbin funciona adecuadamente. El nuevo fichero main.S aparece a continuación:

```
        .text
        .global main

main:
        stmfd sp!,{lr}    @ salva LR por si retornamos
        adr   r0, texto1
        bl    _U0puts
        ldr   r0, =0xFFAA5500
        bl    U0prtbin
        adr   r0, crlf
        bl    _U0puts
        ldmfd sp!,{pc}    @ retornamos

texto1: .asciz "\r\nBienvenidos al LPC2103\r\n--
            Esqueleto de programa en ensamblador --"
crlf:   .asciz "\r\n"

        .align 2

U0prtbin:
        stmfd sp!, {r4, r5, lr}
        mov   r4, r0
        mov   r5, #32
1:      movs  r4, r4, lsl #1
        movcs r0, #'1'
        movcc r0, #'0'
        bl    _U0putch
        subs  r5, r5, #1
        bne   1b
        ldmfd sp!, {r4, r5, pc}

        .end
```

Ejercicio 0: Ejecutar el programa. ¿Se obtiene el resultado esperado? Comentar cada línea de la subrutina indicando su función, de modo que se entienda el algoritmo utilizado.

Ejercicio 1: Modificar el programa **main** para que, tras mostrar el contenido del registro r0 en **binario**, en las líneas siguientes lo muestre en **hexadecimal** y en **decimal**. Añadir entre estos valores unas líneas de texto que expliquen lo que se va a representar:

```
Bienvenidos al LPC2103
-- Esqueleto de programa en ensamblador -
En binario:
11111111101010100101010100000000
En hexadecimal:
FFAA5500
En decimal (sin signo):
4289352960
En decimal (con signo):
-5614336
```

Para mostrar el número en decimal con signo, debéis elaborar una subrutina de nombre **U0prtados** a la que se pasa el número (que puede ser positivo o negativo) en un registro y lo imprime por el hiperterminal.

Ejercicio 2: Realizar un segundo programa que permita la introducción de un carácter ASCII a través del teclado y lo muestre por el hiperterminal en ASCII, en binario, en hexadecimal [2] y en decimal sin signo.

Tabla con el código ASCII:

BITS B4	B3	B2	B1	B7 B6 B5 = 0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
				CONTROL		NUMBERS & SYMBOLS		UPPERCASE		LOWERCASE	
0	0	0	0	NUL 0	DLE 16	SP 32	0 48	@ 64	P 80	\ 96	p 112
0	0	0	1	SOH 1	DC1 17	! 33	1 49	A 65	Q 81	a 97	q 113
0	0	1	0	STX 2	DC2 18	" 34	2 50	B 66	R 82	b 98	r 114
0	0	1	1	ETX 3	DC3 19	# 35	3 51	C 67	S 83	c 99	s 115
0	1	0	0	EOT 4	DC4 20	$ 36	4 52	D 68	T 84	d 100	t 116
0	1	0	1	ENQ 5	NAK 21	% 37	5 53	E 69	U 85	e 101	u 117
0	1	1	0	ACK 6	SYN 22	& 38	6 54	F 70	V 86	f 102	v 118
0	1	1	1	BEL 7	ETB 23	' 39	7 55	G 71	W 87	g 103	w 119
1	0	0	0	BS 8	CAN 24	(40	8 56	H 72	X 88	h 104	x 120
1	0	0	1	HT 9	EM 25	) 41	9 57	I 73	Y 89	i 105	y 121
1	0	1	0	LF 10	SUB 26	* 42	: 58	J 74	Z 90	j 106	z 122
1	0	1	1	VT 11	ESC 27	+ 43	; 59	K 75	[91	k 107	{ 123
1	1	0	0	FF 12	FS 28	, 44	< 60	L 76	\ 92	l 108	| 124
1	1	0	1	CR 13	GS 29	– 45	= 61	M 77	] 93	m 109	} 125
1	1	1	0	SO 14	RS 30	. 46	> 62	N 78	↑ 94	n 110	~ 126
1	1	1	1	SI 15	US 31	/ 47	? 63	O 79	_ 95	o 111	↓ 127

[2] Nótese que el carácter ASCII introducido por el teclado ocupa 1 byte, que será el byte menos significativo del registro que lo contenga. En el caso de la visualización en binario y en hexadecimal, se tratará como un dato de 32 bits y no será necesario eliminar los ceros a la izquierda.